

不确定性下多处理器调度的近似算法

Guolong Lin*

Akamai Technologies
8 Cambridge Center, Cambridge, MA 02142
glin@akamai.com

Rajmohan Rajaraman

College of Computer and Information Science
Northeastern University, Boston MA 02115
rraj@ccs.neu.edu

摘要

受网络计算和项目管理应用的启发，我们研究把作业分配给处理器后，作业能否成功执行存在不确定性的多处理器调度场景。我们考虑不确定性下的多处理器调度问题：给定 n 个单位时间作业、 m 台机器、表示作业间依赖关系的有向无环图 C ，以及每个作业 j 和机器 i 对应的概率 p_{ij} ，表示任意特定时间步把作业 j 安排到机器 i 后成功完成的概率。目标是寻找使期望完工跨度最小的调度，即使全部作业完成所需时间的期望最小。

不确定性下的多处理器调度问题由 Malewicz 提出，并且已经证明，即使全部作业相互独立，该问题仍为 NP 难。本文针对依赖 DAG C 的若干特殊情形，给出多项式时间近似算法：独立作业情形得到 $O(\log n)$ 近似； C 是互不相交的链集合时，得到 $O(\log m \log n \log(n+m) / \log \log(n+m))$ 近似； C 是有向出树或入树集合时，得到 $O(\log m \log^2 n)$ 近似； C 是有向森林时，得到 $O(\log m \log^2 n \log(n+m) / \log \log(n+m))$ 近似。

类别与主题描述

F.2 [计算理论]: 算法分析

一般术语

算法，理论

关键词

近似算法，多处理器调度

1. 引言

我们研究不确定性下的多处理器调度问题。该问题由 [21] 提出，用来研究将作业分配给服务器后，能否成功完成存在不确定性的场景。一个动机应用是网络计算：大量通常分布在不同地理位置的计算机协作求解复杂计算任务。为更好地利用这些分布式计算机，通常把一个任务划分为较小部分（即作

*本工作的部分内容在作者任职于 Northeastern University 期间完成。

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

SPAA'07, June 9–11, 2007, San Diego, California, USA.
Copyright 2007 ACM 978-1-59593-667-7/07/0006 ...\$5.00.

业)，交给不同计算机处理。在许多应用中，这些作业之间存在并不简单的依赖关系。由于可能发生物理故障，或仅由于计算环境的分布式特性，机器可能无法按时成功执行被分配的作业。此时，自然的目标是确定向计算机分配给定作业的调度，使任务的期望完成时间最小。

另一个类似例子是组织中的大型项目管理，[21] 也讨论了这一场景。项目可以拆分为存在依赖关系的小作业，即一个作业只有在另一组作业成功完成后才能执行。项目配有一组工作者。受实际因素和技能差异影响，工作者可能无法按时成功完成分配给自己的作业。为降低某些关键作业潜在延迟的概率，项目经理可以，而且通常也希望，同时给这些作业分配多名工作者。根据过去的经验和工作者的技能水平，项目经理可以估计任一工作者完成任一作业的成功概率。经理面临的挑战是设计工作者到作业的分配策略或调度，使整个项目的期望完成时间尽可能小。

受上述例子启发，我们研究不确定性下的多处理器调度，下文称为 SUU。给定 m 台机器、 n 个单位时间作业，以及表示作业执行先后约束的有向无环图。此外，对每个作业 j 和机器 i ，给定任意特定时间步把作业 j 安排到机器 i 后成功完成的概率 p_{ij} 。为弥补这种不确定性，可以同时把多台机器分配给同一作业。我们关注如何计算一个调度，使全部作业完成所需时间的期望最小，也就是最小化期望完工跨度。

1.1 本文结果

[21] 已证明，即使全部作业相互独立，多处理器调度问题 SUU 仍是 NP 难的。本文针对几类特殊的依赖图，给出 SUU 的近似算法。

- 首先考虑全部作业相互独立的情形，给出 $O(\log n)$ 近似算法 (§3)。

处理独立作业情形的关键，是构造一个使各作业成功概率之和最大的子问题。对类似策略进行细化，使其能够处理作业依赖后，就可以求解作业并不独立的更一般情形。

- 当作业的先后约束构成一组互不相交的链时，我们在 §4.1 得到一个 $O(\log m \log n \frac{\log(n+m)}{\log \log(n+m)})$ 近似算法。结果依赖于求解一个松弛线性规划，并利用网络流理论的结果对分数解进行舍入。
- 利用互不相交的链的算法，以及 [17] 的链分解技术，分别针对入树或出树集合、有向森林得到 $O(\log m \log^2 n)$ 和 $O(\log m \log^2 n \frac{\log(n+m)}{\log \log(n+m)})$ 近似 (§4.2)。

针对互不相交的链、树和有向森林的算法所计算的调度，都是无反馈的：它们预先规定每个时间步机器到作业的分配，而不

依赖于当时尚未完成的作业集合。无反馈调度在 §2 中正式定义；该节还给出主要结果所用的相关定义和重要调度性质。

据我们所知，这些结果是不确定性下多处理器调度问题的首批近似算法。

1.2 相关工作

本文研究的问题最早由 Malewicz [21] 定义，主要受到网格计算中复杂 DAG 调度应用的启发 [9]。Malewicz 根据机器数量和依赖图的宽度刻画问题复杂度，其中宽度定义为独立作业的最大数量。他证明，当机器数量和宽度都是常数时，可以用动态规划在多项式时间内计算最优调度方案。但任一参数不受界限限制时，问题就是 NP 难的。此外，除非 P=NP，否则问题无法在 $5/4$ 因子内得到近似解。本文通过研究 DAG 宽度和机器数量都不受界限限制时的可近似性，扩展了 Malewicz 的工作。

本文调度问题的不确定性来自机器执行所分配作业时可能失败，并通过 p_{ij} 建模。调度文献中还有其他不确定性模型，最突出的是每项任务的持续时间为随机长度、且可能需要不同数量资源的模型。相关工作参见 [7, 6, 14, 29, 16, 11]。

调度研究有悠久的历史 and 大量文献。依据不同因素，调度问题有很多变体。例如：机器是否相关？执行是否允许抢占？作业执行是否有先后约束？作业是否具有释放时间？目标函数是完工跨度、加权完成时间，还是加权流时间等？综述参见 [13]，代表性工作参见 [12, 20, 28, 19, 4, 17]。

与本文密切相关的两个具体变体，是作业车间调度 [27] 和具有先后约束的不相关机器调度。作业车间调度中，给定 m 台机器和 n 个作业，每个作业包含一系列操作。每个操作必须由指定机器处理，作业需要按照给定顺序依次处理其操作。在任意时刻，一台机器至多处理一个作业。目标是在机器上安排这些作业，使最大完成时间最小。该问题是强 NP 难的，已有大量研究 [10, 18, 1]。另一个得到广泛研究的问题，是在不相关并行机器上，以抢占方式调度具有先后约束的作业 [19, 27, 17]；作业处理时间取决于它被分配到的机器。该问题与 SUU 的共同特点是，机器 i 完成作业 j 的能力同时随 i 和 j 变化。不相关并行机器问题通过随 i 和 j 变化的确定性处理时间建模这种非均匀性；在 SUU 中，作业大小都为单位值，但未能完成的概率随 i 和 j 变化。由于作业完成存在不确定性，SUU 调度似乎更难表示和分析。另一个技术区别是，SUU 允许同时把多台机器分配给同一作业，以提高该作业成功完成的概率。不相关并行机器问题通常通过归约为作业车间调度实例来求解。本文的一些 SUU 算法也使用类似归约。

2. 调度、成功概率与质量

本节正式定义调度 (§ 2.1)，引入作业的质量概念，并证明一个关键技术定理：给定调度的期望完工跨度以内，作业能够积累一定的质量 (§ 2.2)。

2.1 调度

在 SUU 中，给定包含 n 个单位时间步作业的集合 J ，以及包含 m 台机器的集合 M 。作业之间有先后约束，形成有向无环图 (DAG) C 。如果根据先后约束，作业 j 的所有前驱作业都在 t 之前成功完成，则作业 j 在时间步 t 是可执行的。对于每个作业 j 和机器 i ，还给定概率 p_{ij} ，表示将作业 j 安排到机器 i 后能够成功完成的概率，而且与其他任意执行结果独立。同一时间步可以把多台机器分配给同一作业。不失一般性，假设对每个 j ，至少存在一台机器 i 满足 $p_{ij} > 0$ 。

定义 2.1. 长度为 $T \in \mathbb{Z}^+ \cup \{\infty\}$ 的调度 Σ ，是函数集合 $\{f_{S,t} : M \rightarrow J \cup \{\perp\} \mid S \subseteq J, 1 \leq t \leq T+1\}$ 。调度 Σ 的一次执行是指：每个时间步 t 开始时，若 S 是尚未完成的作业

集合，则当 $f_{S,t}(i)$ 可执行且属于 S 时，把机器 i 分配给作业 $f_{S,t}(i)$ ；否则，机器 i 在该时间步空闲。

调度的正式定义为无限多个 t 指定分配函数 $f_{S,t}$ 。这是因为，若 $\forall i, p_{ij} < 1$ ，那么在任意给定时间步后，作业 j 仍有正概率尚未完成。不过，在优化期望完工跨度时，可以只考虑某个受限制的调度类别。

定义 2.2 ([21]). 平稳调度方案 Σ_g 是满足如下条件的调度：对于任意 $S \subseteq J$ 和 $t_1 \neq t_2$ ，都有 $f_{S,t_1}(\cdot) = f_{S,t_2}(\cdot)$ 。换言之，分配函数 $f_{S,t}$ 只依赖尚未完成的作业集合 S 。因此，可以用完整函数集合 $\{f_S : M \rightarrow S \cup \{\perp\} \mid S \subseteq J\}$ 表示 Σ_g 。

将给定 SUU 实例的最小期望完工跨度记为 T^{OPT} 。它是有限的，因为对任意作业 j ，都存在机器 i 满足 $p_{ij} > 0$ 。不难看出，存在一个最优调度，它同时也是平稳调度方案：在任意时间步 t ，都可以确定一个最优分配函数，该函数只依赖时间步 t 开始时尚未完成的作业子集，而与过去的执行历史和 t 的具体值无关。虽然直接表示任意平稳调度方案需要 2^n 个不同的分配函数，但某些方案可以简洁表示，例如用一个长度为多项式的函数，以 S 为输入并返回 f_S 。本文还考虑另一类受限制的调度，称为无反馈调度。

定义 2.3. 无反馈调度是一种每个分配函数 $f_{S,t}$ 都与 S 无关的调度，即对所有 t, S, S' ，都有 $f_{S,t}(\cdot) = f_{S',t}(\cdot)$ 。因此，任意时间步 t 的分配都可以用一个函数表示，将其记为 f_t 。

无反馈调度有两个吸引人的特点。首先，在任意时间步 t ，无论 t 时实际出现的未完成作业集合 S 是什么，都只需要一个分配函数。回顾一下，执行的不确定性会使给定 t 可能对应许多个不同的 S 。其次，一个更偏技术层面的益处是，无反馈调度允许通过求解相关的确定性优化问题，处理 SUU 问题中的不确定性。

2.2 成功概率与质量

在任意时间步，将机器子集 $S \subseteq M$ 分配给 j 时， j 成功完成的概率是 $1 - \prod_{i \in S} (1 - p_{ij})$ 。为便于近似分析，我们使用下面的命题。

命题 2.1. 给定 $x_1, \dots, x_k \in [0, 1]$ ，有 $1 - (1 - x_1) \cdots (1 - x_k) \leq x_1 + \dots + x_k$ 。进一步，若 $x_1 + \dots + x_k \leq 1$ ，则 $1 - (1 - x_1) \cdots (1 - x_k) \geq e^{-1}(x_1 + \dots + x_k)$ 。

证明。第一个结论来自恒等式 $(1 - x_1) \cdots (1 - x_k) \geq 1 - (x_1 + \dots + x_k)$ ，可以用简单归纳证明。基例 $k = 1$ 是平凡的。假设该恒等式对于 $k - 1$ 成立。如果 $x_1 + \dots + x_{k-1} > 1$ ，则它对于 k 成立；否则，根据归纳假设，

$$\begin{aligned} & (1 - x_1) \cdots (1 - x_{k-1})(1 - x_k) \\ & \geq [1 - (x_1 + \dots + x_{k-1})](1 - x_k) \\ & \geq 1 - (x_1 + \dots + x_k). \end{aligned}$$

对于第二个结论，注意若 $0 \leq x \leq 1$ ，则 $1 - x \leq e^{-x} \leq 1 - \frac{x}{e}$ 。因为 $1 - x \leq e^{-x}$ ，所以 $(1 - x_1) \cdots (1 - x_k) \leq e^{-x_1} \cdots e^{-x_k}$ ，从而有

$$\begin{aligned} & 1 - (1 - x_1) \cdots (1 - x_k) \\ & \geq 1 - e^{-x_1} \cdots e^{-x_k} \\ & = 1 - e^{-(x_1 + \dots + x_k)} \\ & \geq \frac{x_1 + \dots + x_k}{e}, \end{aligned}$$

最后一个不等式来自对 $x \in [0, 1]$ 有 $e^{-x} \leq 1 - \frac{x}{e}$ ，以及假设 $x_1 + \dots + x_k \leq 1$ 。

□

命题 2.1 表明, 可以用一个方便的线性形式近似成功概率。

定义 2.4. 对任意调度 Σ , 将作业 j 在时间步 t 结束时的质量定义为: 对全部时间 $t' \in [1, t]$, 以及在时间 t' 被分配给 j 的每台机器 i , 累加 p_{ij} 。因此, 对任意调度, 作业 j 在时间 t 的质量是随机变量。对于无反馈调度 Σ_o , j 在任意时间步 t 结束时的质量就是

$$\min\left\{\sum_{1 \leq \tau \leq t} \sum_{i: f_\tau(i)=j} p_{ij}, 1\right\},$$

其中, $f_\tau(\cdot)$ 是 Σ_o 在时间步 τ 的分配函数。称 j 到时间步 t 已经积累了该质量。

下面的定理对我们的调度方法至关重要。强调一下, 它对任意 SUU 实例都成立, 并用于定理 3.1 和引理 4.2 的证明。

定理 2.2. 令 Σ 为某个 SUU 实例的调度, 其期望完工跨度为 T 。对任意作业 j , 执行 Σ 共 $2T$ 步后, j 以至少 $1/4$ 的概率积累至少 $1/4$ 的质量。

证明。令 A 表示 j 在时间步 $2T$ 以内完成的事件。令 S_t 为时间步 t 分配给作业 j 的机器集合这一随机变量, 并令 $P(S_t) = \sum_{i \in S_t} p_{ij}$ 。令 B 表示 $\sum_{1 \leq t \leq 2T} P(S_t) \leq 1/4$ 这一事件。需要证明的是 $\Pr(B^c) \geq 1/4$ 。注意, $\Pr(A)$ 等于 $\Pr(A \cap B) + \Pr(A \cap B^c)$, 而后者至多为 $\Pr(A \cap B) + \Pr(B^c)$ 。

下面估计 $\Pr(A \cap B)$ 。注意, Σ 在这些作业上的全部可能执行构成一棵无限有根树, 其中每个节点代表执行中的一个中间状态 (图 1 给出示意)。每个节点关联一组作业, 表示该状态下尚未完成的作业。对节点 N , 令 $\text{Jobs}(N)$ 为其关联的未完成作业集合。注意, 第 0 层根节点 R 的 $\text{Jobs}(R)$ 包含全部作业。第 k 层节点表示执行 k 步后的状态。从第 k 层节点 N 到第 $k+1$ 层节点 Q , 可以根据分配函数 $f_{\text{Jobs}(N), k+1}$ 计算相应的转移概率。

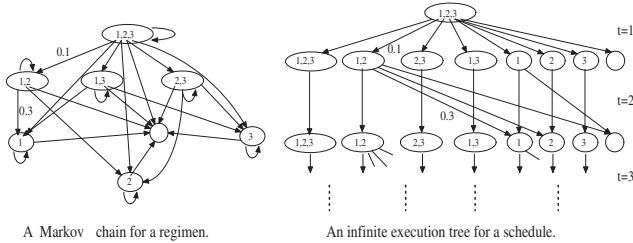


图 1: 调度示意。为简单起见, 只使用 3 个作业。每个节点表示一个中间状态, 内部显示其关联的未完成作业集合。边旁的数字表示转移概率。左图是平稳调度方案的 Markov 链表示, 右图是调度执行的有根树表示。为避免拥挤, 仅对时间步 2 的节点 $\{1, 2\}$ 和 $\{1\}$ 显示全部转移。

引理 2.3. 考虑第 k 层树节点 N , 其中 $j \in \text{Jobs}(N)$ 。对 $1 \leq t \leq k$, 令 S_t 为沿根节点 R 到 N 的路径, 在时间步 t 分配给 j 的机器集合。假设 $\sum_{1 \leq t \leq k} P(S_t) \leq c$, 其中 $c \leq 1$ 。令 $P(j, N)$ 表示如下事件的概率: 沿一条经过 N 的树路径, j 在第 $2T$ 层 (时间步) 之前完成, 且 $\sum_{1 \leq t \leq 2T} P(S_t) \leq c$ 。那么, $P(j, N) \leq c - \sum_{1 \leq t \leq k} P(S_t)$ 。

引理的证明: 对层号 k 进行反向归纳。先考虑基例: N 位于第 $2T-1$ 层, 只需再执行一个时间步。令 S_{2T} 为时间步 $2T$ 分配给 j 的机器集合。若 $P(S_{2T}) > c - \sum_{1 \leq t \leq 2T-1} P(S_t)$, 则

$P(j, N) = 0$ 。否则, j 在这一步内完成的概率至多为 $P(S_{2T})$ 。无论哪种情形, 断言都成立。

现在假设断言对任意第 $k \leq 2T-1$ 层成立, 目标是证明其对第 $k-1$ 层也成立。考虑第 $k-1$ 层的树节点 N 。令 S_k 为按照分配函数 $f_{\text{Jobs}(N), k}$, 在时间步 k 分配给 j 的机器集合。 N 的第 k 层子节点或者不包含 j (j 在时间步 k 完成), 或者包含 j (j 在时间步 k 尚未完成)。将这两种情形的概率分别记为 P_1 和 $1 - P_1$ 。将所有 j 尚未完成的子节点记为 L 。

若 $P(S_k) > c - \sum_{1 \leq t \leq k-1} P(S_t)$, 则 $P(j, N) = 0$, 它满足 $\leq c - \sum_{1 \leq t \leq k-1} P(S_t)$ 。否则,

$$\begin{aligned} P(j, N) &= P_1 + \sum_{Q \in L} P(j, Q) \\ &\leq P_1 + \sum_{Q \in L} (c - \sum_{1 \leq t \leq k} P(S_t)) \\ &= P_1 + (1 - P_1)(c - \sum_{1 \leq t \leq k} P(S_t)) \\ &\leq P_1 + (c - \sum_{1 \leq t \leq k} P(S_t)) \\ &\leq c - \sum_{1 \leq t \leq k-1} P(S_t), \end{aligned}$$

第二个不等式来自归纳假设, 最后一个不等式来自 $P_1 \leq P(S_k)$ 。这证明了归纳步骤, 因此引理成立。

□

取 $c = 1/4$ 并应用引理, 得到 $\Pr(A \cap B) = P(j, R) \leq c = 1/4$ 。因此, $\Pr(A) \leq 1/4 + \Pr(B^c)$ 。由 Markov 不等式, $\Pr(A) \geq 1/2$ 。所以 $\Pr(B^c) \geq 1/4$, 证明完成。

3. 独立作业

本节研究调度问题中作业相互独立这一特殊情形, 将该问题记为 SUU-I。为计算 SUU-I 的解, 首先证明存在一个无反馈调度, 使作业在 $O(T^{\text{OPT}})$ 步内积累的总质量为 $\Omega(n)$ 。为寻找这样的调度, 我们构造一个最大化质量总和的子问题, 随后给出多项式时间算法, 计算 SUU-I 的 $O(\log n)$ 近似调度和 $O(\log^2 n)$ 近似无反馈调度。对于无反馈调度, §4.1 研究更一般的链式先后约束时, 将近似因子改进为 $O(\log n \cdot \log(\min\{n, m\}))$ 。

定理 3.1. 若 SUU-I 存在期望完工跨度为 T 的调度 Σ , 则存在长度为 $2T$ 的无反馈调度, 其中全部作业积累的总质量至少为 $n/16$ 。

证明。考虑 Σ 执行 $2T$ 步的一次执行 E 。这自然给出长度为 $2T$ 的无反馈调度 Σ_E , 分配函数 $f_t(\cdot)$ 定义如下: 若在 E 的时间步 t 中, 机器 i 被分配给作业 j , 则 $f_t(i) = j$ 。注意, 由于执行的不确定性, E 以及由其确定的 Σ_E 都是随机变量。由定理 2.2, 对任意作业 j , 在 Σ_E 的时间步 $2T$ 之前, j 以至少 $1/4$ 的概率积累至少 $1/4$ 的质量。因此, Σ_E 的时间步 $2T$ 时, j 的期望质量至少为 $1/16$ 。这意味着, 在 Σ_E 的时间步 $2T$ 时, 全部作业的期望总质量至少为 $n/16$ 。所以, 存在一个无反馈调度, 使全部作业到时间步 $2T$ 的总质量至少为 $n/16$ 。

□

3.1 SUU-I 的 $O(\log n)$ 近似调度

受定理 3.1 启发, 我们构造最大化质量之和的子问题 MaxSumMass。在 MaxSumMass 中, 给定包含 n 个独立单位时间步作业的集合 J 、包含 m 台机器的集合 M , 以及概率 p_{ij} ; 目标是为单个时间步寻找分配 $f: M \rightarrow J \cup \{\perp\}$, 使该步各

作业的质量之和最大。图 2 给出 MaxSumMass 的 1/3 近似算法 MSM-ALG (可以证明该问题是 NP 难的), 以及我们的 SUU-I 近似算法, 后者只需在每个时间步对未完成作业执行 MSM-ALG。

定理 3.2. MSM-ALG 为问题 MaxSumMass 计算一个 1/3 近似解。□

证明。考虑一个二部图, 一侧是作业 J 的节点, 另一侧是机器 M 的节点。对任意 $p_{ij} > 0$, 机器 i 和作业 j 之间有一条边 (i, j) 。可以把 MSM-ALG 看作选择边并确定方向。令 $\text{Opt} = \{(i, j)\}$ 为最优分配 f^* 选择的边集合, Sol 为 MSM-ALG 计算的解。下面使用记账论证。考虑任意边 $(i, j) \in \text{Opt}$ 。

1. $(i, j) \in \text{Sol}$: 将 p_{ij} 记账给自身。
2. $(i, j) \notin \text{Sol}$:
 - (a) 在第 2 步中, 因为 $f(i) \neq \text{nil}$ 而未加入 (i, j) 。令 $j' = f(i)$ 。将 p_{ij} 记账给 $p_{ij'}$, 其中 $(i, j') \in \text{Sol}$ 。注意, $p_{ij} \leq p_{ij'}$; 由于 Opt 中每台机器 i 至多使用一次, $p_{ij'}$ 在这种情形下至多被记账一次。
 - (b) 在第 2 步中, 因为 $f(i) = \text{nil}$ 但 $\sum_{x: f(x)=j} p_{xj} + p_{ij} > 1$ 而未加入 (i, j) 。由于按降序处理 p_{ij} , 可知在 Sol 中有 $\sum_{x: f(x)=j} p_{xj} \geq 1/2$ 。将 p_{ij} 记账给 $2 \sum_{x: f(x)=j} p_{xj}$ 。

注意, 一份 Sol 足以覆盖类型 1 和类型 2(a) 的记账; 两份 Sol 足以覆盖类型 2(b) 的记账, 因为根据定义, 任意分配中每个作业的质量至多为 1。

因此, MSM-ALG 计算的解具有 1/3 的近似因子。

□

定理 3.3. 算法 SUU-I-ALG 是 SUU-I 的 $O(\log n)$ 近似算法。

证明。令 S_t 表示时间步 t 开始时尚未完成的作业集合。由定理 3.1, 存在一个从时间步 t 开始、长度为 $2T^{\text{OPT}}$ 的无反馈调度, 其中 S_t 的全部作业积累的总质量至少为 $|S_t|/16$ 。对该调度的 $2T^{\text{OPT}}$ 个时间步取平均, 可知时间步 t 存在一个作业到机器的分配, 使 S_t 中作业在时间步 t 积累的总质量至少为 $|S_t|/(32T^{\text{OPT}})$ 。由定理 3.2, 在 SUU-I-ALG 的时间步 t , 作业在时间步 t 积累的总质量至少为 $|S_t|/(96T^{\text{OPT}})$ 。由命题 2.1, 时间步 t 完成的作业数的期望至少为 $|S_t|/(96eT^{\text{OPT}})$ 。

于是, 随机变量序列 S_t 满足 $E[|S_{t+1}| | S_t] = |S_t|(1 - 1/(96eT^{\text{OPT}}))$ 。通过直接的 Chernoff 界论证 [3, 15], 可得在 $O(T^{\text{OPT}} \log n)$ 步以内, S_t 以高概率变为空集。

□

3.2 SUU-I 的近似无反馈调度

SUU-I-ALG 计算的调度是自适应的: 每一步的分配函数依赖于该步开始时尚未完成的作业集合。本节通过扩展 MSM-ALG, 设计一个多项式时间组合算法, 计算期望完工跨度在最优值 $O(\log^2 n)$ 倍以内的无反馈调度。§4.1 将使用基于线性规划的算法, 进一步把该界改进为 $O(\log n \cdot \log(\min\{n, m\}))$ 。

由定理 3.1, 存在长度为 $2T^{\text{OPT}}$ 的无反馈调度, 其中全部作业的总质量至少为 $n/16$ 。直观而言, 如果计算一个长度为 $2T^{\text{OPT}}$ 的无反馈调度 Σ_1 , 目标是最大化各作业质量的总和, 那么应当有许多作业在 Σ_1 中积累常数质量。随后可以移除这些作业, 并计算另一个长度为 $2T^{\text{OPT}}$ 的无反馈调度 Σ_2 , 最大化剩余作业的质量总和, 再移除一些积累了常数质量的作业。

因为每次计算无反馈调度都会移除许多作业, 该过程应当很快终止。将 $\Sigma_1, \Sigma_2, \dots$ 依次连接, 就得到一个无反馈调度 Σ , 其中每个作业都积累常数质量。

由定理 3.2, 问题 MaxSumMass 有一个 1/3 近似算法。不过, MaxSumMass 只考虑长度为 1 的无反馈调度, 即每台机器至多被分配给一个作业。我们需要的是寻找长度为 $2T^{\text{OPT}}$ 的无反馈调度, 并最大化各作业质量之和的过程。事实表明, 很容易扩展 MSM-ALG, 使其考虑任意调度长度, 同时保持 1/3 的近似因子。下面正式表述。

问题 (MaxSumMass-Ext): 给定包含 n 个独立单位时间步作业的集合 J , 以及包含 m 台机器的集合 M 。令 p_{ij} 表示把作业 j 分配给机器 i 后成功完成的概率。还给定参数 $t \in \mathbb{Z}^+$ 。目标是寻找长度为 t 的无反馈调度 Σ_o , 使各作业到时间步 t 积累的质量总和最大。

下面给出算法 MSM-E-ALG, 它输出一个长度为 $t \in \mathbb{Z}^+$ 的无反馈调度 Σ_o , 是问题 MaxSumMass-Ext 的 1/3 近似解。MSM-E-ALG 是对 MSM-ALG 的简单修改: 因为调度长度为 t , 每台机器可以分配 t 次。为每台机器维护一个初始值为 t 的剩余容量参数 t_i , 记录机器 i 还能在哪些时间步得到分配。还使用 x_{ij} 记录机器 i 被分配给作业 j 的时间步数。在 MSM-E-ALG 的步骤 2(a) 中, 只要 t_i 为正, 就尽可能多地把 i 分配给 j ; 步骤 2(b) 相应更新 t_i 。步骤 3 输出无反馈调度 $\Sigma_o = \{f_\tau(\cdot) : 1 \leq \tau \leq t\}$, 可以通过 x_{ij} 表示如下。令 $j_1, \dots, j_n$ 为作业排序; 对 $1 \leq k \leq n$, 当 $\sum_{1 \leq l < k} x_{ij_l} + 1 \leq \tau \leq \sum_{1 \leq l \leq k} x_{ij_l}$ 时, $f_\tau(i) = j_k$ 。注意, MSM-E-ALG 的运行时间与 t 的值无关, 因为每个 p_{ij} , 以及由其对应的每一对 (i, j) , 都在步骤 2 中恰好处理一次。套用定理 3.2 中对 MSM-ALG 的类似分析, 不难看出 MSM-E-ALG 为问题 MaxSumMass-Ext 输出一个 1/3 近似解。

算法 1 MSM-E-ALG

输入: 作业 J , 机器 M , p_{ij} 和 t_o

1. 按降序排列 p_{ij} 。初始化: $\forall i, t_i \leftarrow t; \forall i, j, x_{ij} \leftarrow 0$ 。
 2. 按排序依次处理每个 p_{ij} :
 - (a) $x_{ij} \leftarrow \min \left\{ t_i, \left\lfloor \frac{1 - \sum_{k \in M} x_{kj} \cdot p_{kj}}{p_{ij}} \right\rfloor \right\}$ 。
 - (b) $t_i \leftarrow t_i - x_{ij}$ 。
 3. 输出由 x_{ij} 指定的 Σ_o 。
-

引理 3.4. MSM-E-ALG 为问题 MaxSumMass-Ext 计算一个近似因子为 1/3 的解。

现在给出问题 SUU-I 的近似算法 SUU-I-OBL。

说明 SUU-I-OBL 的几个要点。我们反复使用 MSM-E-ALG, 每轮让相当一部分作业积累常数质量, 直到全部作业都积累常数质量。不过还存在一个障碍: 不知道 T^{OPT} 的值, 因此必须为 MSM-E-ALG “猜测” 一个 t 值。它必须足够大, 例如至少为 $2T^{\text{OPT}}$, 才能保证存在长度为 t 、总质量至少为 $n/16$ 的无反馈调度, 如定理 3.1 所证明的那样。概括而言, 在 SUU-I-OBL 的循环 (步骤 3) 中, 反复调用 MSM-E-ALG, 让作业积累 1/96 的质量, 至多进行 $66 \log n$ 轮 (稍后说明原因)。循环结束 (步骤 4) 时, 若仍有剩余作业, 则说明 t 值还不够大。因此, 将 t 翻倍, 重置其他参数, 再尝试新的 t 。注意, 每次调用 MSM-E-ALG 时都会从零开始, 忽略这些作业在前几轮可能积累的任何质量。下面分析 SUU-I-OBL 的性能。

若 $t \geq 2T^{\text{OPT}}$, 以 t 调用一次 MSM-E-ALG, 令 x 为得到至少 1/96 质量的作业数量。全部作业的质量总和至多为

算法 MSM-ALG**输入:** 作业 J , 机器 M , p_{ij} 。

- 将 $f(i)$ 置为 nil, $i \in M$ 。
- 按非增序依次处理每个 p_{ij} : 若 $f(i)$ 为 nil 且 $\sum_{x:f(x)=j} p_{xj} + p_{ij} \leq 1$, 则将 i 分配给 j , 即 $f(i) \leftarrow j$ 。
- 对每台未使用的机器 i , 令 $f(i) \leftarrow \perp$; 输出 f 。

算法 SUU-I-ALG**输入:** 作业 J , 机器 M , p_{ij} 。

- 令 S_t 表示时间步 t 开始时尚未完成的作业集合。
- 在每个时间步 t , 将 MSM-ALG 应用于 S_t 和全部机器, 按其确定的分配进行调度。

图 2: 独立作业调度的近似算法。**算法 2 SUU-I-OBL****输入:** 作业 J , 机器 M , p_{ij} 。

1. $t \leftarrow 1$.
2. $I \leftarrow 1$. $R \leftarrow J$. $\Sigma \leftarrow$ “空调度”。
3. 当 $(|R| > 0)$ 且 $(I \leq 66 \log n)$ 时, 重复以下操作:
 - (a) 令 Σ_I 为以当前 t 值、在 R, M 上调用 MSM-E-ALG 的输出。 $\Sigma \leftarrow \Sigma \circ \Sigma_I$ 。
 - (b) 从 R 中移除积累了至少 $1/96$ 质量的作业。
 - (c) $I \leftarrow I + 1$.
4. 若 $|R| > 0$, 则令 $t \leftarrow 2t$, GOTO 步骤 2; 否则返回 Σ 。

$x \cdot 1 + (n - x) \cdot 1/96$, 因为任意作业积累的质量至多为 1。由定理 3.1, 存在长度为 t 、总质量至少为 $n/16$ 的无反馈调度。由引理 3.4, MSM-E-ALG 的近似比为 $1/3$ 。因此,

$$x \cdot 1 + (n - x) \cdot 1/96 \geq 1/3 \cdot n/16.$$

由此有 $x \geq n/95$ 。因为每次调用 MSM-E-ALG 都会让至少 $1/95$ 的作业积累 $1/96$ 的质量, 所以至多调用 MSM-E-ALG 共 $66 \log n$ 次, 就足以让全部作业积累至少 $1/96$ 的质量。

为证明 SUU-I-OBL 在多项式时间内终止, 首先给出 T^{OPT} 的界。令 $p_{\min} = \min_{i,j} p_{ij}$ 。显然, 若任意时间步都把全部机器分配给单个作业, 让作业逐个积累足够质量, 则每个作业在长度为 $\lceil \frac{n}{p_{\min}} \rceil$ 的时间区间内都积累至少 1 的质量。这意味着 $T^{\text{OPT}} = O(\frac{n}{p_{\min}} \log n)$ 。由于 SUU-I-OBL 每次迭代都将 t 翻倍, 算法终止前将“探测” $O(\log n + \log \frac{1}{p_{\min}})$ 个不同的 t 值。对每个 t 值, 至多调用 MSM-E-ALG 共 $66 \log n$ 次, 而每次调用都在多项式时间内运行。因此, SUU-I-OBL 在相对于输入大小为多项式的时间内终止。我们已证明:

引理 3.5. 对于问题 SUU-I, 可以在多项式时间内计算长度为 $O(\log n)T^{\text{OPT}}$ 的无反馈调度, 使每个作业积累至少 $1/96$ 的质量。

定理 3.6. 对于问题 SUU-I, 可以在多项式时间内计算一个无反馈调度, 其期望完工跨度在最优值的 $O(\log^2 n)$ 倍以内。

证明。利用引理 3.5, 首先计算长度为 $T = O(\log^2 n) \cdot T^{\text{OPT}}$ 的无反馈调度 Σ_0 , 使每个作业积累至少 $1/96$ 的质量。 Σ_0 的无限重复 Σ_∞ 就是所需的无反馈调度。将 Σ_∞ 在每个时间步区间 $[k \cdot T + 1, (k+1) \cdot T]$ (其中 $k = 0, 1, \dots$) 的执行视为一次迭代, 由命题 2.1, 每个作业在每次迭代期间的成功概率至少为 $\frac{1}{24e}$ 。经过 $O(\log n)$ 次迭代, 全部作业以高概率完成。因此, Σ_∞ 的期望完工跨度在 T^{OPT} 的 $O(\log^2 n)$ 倍以内。现在正式表述这一论证。

令随机变量 X 为全部作业完成时的迭代次数。下面给出 X 的期望上界。

$$\begin{aligned}
 E[X] &= \sum_{i=0}^{\infty} \Pr(X > i) \\
 &= \sum_{i=0}^{362 \log n - 1} \Pr(X > i) + \sum_{i=362 \log n}^{\infty} \Pr(X > i) \\
 &\leq 362 \log n \cdot 1 + \sum_{i=362 \log n}^{\infty} n \cdot \left(1 - \frac{1}{96e}\right)^i \\
 &= 362 \log n + n \cdot \left(1 - \frac{1}{96e}\right)^{362 \log n} \cdot \sum_{i=0}^{\infty} \left(1 - \frac{1}{96e}\right)^i \\
 &\leq 362 \log n + \frac{96e}{n},
 \end{aligned}$$

第三个不等式来自每个作业在每次迭代以内的成功概率为 $\frac{1}{96e}$; 最后一个不等式来自对几何级数求和, 以及 $(1 - \frac{1}{96e})^{181} < 1/2$ 这一事实。定理证明完成。

□

4. 具有先后约束的作业

本节研究作业具有非平凡先后约束的 SUU。先在 §4.1 为约束构成互不相交的链的情形给出多对数近似算法, 再在 §4.2 扩展到约束构成有向森林的更一般情形。我们计算的全部调度都是无反馈的。

4.1 互不相交的链

考虑 SUU 的一个特殊情形: 作业依赖图 C 是互不相交的链的集合 $C = \{C_1, \dots, C_l\}$ 。将该问题记为 SUU-C。若按约束, 作业 j_1 先于 j_2 , 则写作 $j_1 \prec j_2$ 。

总体而言, 求解 SUU-C 的方法是先计算一个长度接近最优值的无反馈调度, 使每个作业都有常数概率成功完成, 再将该调度复制足够多次, 从而保证全部作业以高概率在所需完工跨度界内完成。首先考虑为每个作业积累常数成功概率的问题。与独立作业情形一样, 使用质量概念代替真实概率。但这里必须考虑作业之间的依赖。因此, 构造问题 AccuMass-C: 给定 SUU-C 的输入, 计算满足两个条件、长度 T 最小的无反馈调度: (i) 每个作业 j 在 T 以内积累至少 $1/2$ 的质量; (ii) 若 $j_1 \prec j_2$, 则必须先让 j_1 积累质量 $1/2$, 才能给 j_2 分配任何机器。条件 (ii) 的直观含义是, 若 j_1 在时间步 t 之前成功完成的概率很低, 则 j_2 在时间步 t 可执行的概率也很小; 因此, 在无反馈调度中于 t 之前给 j_2 分配机器没有太大意义。

下面给出

AccuMass-C 的松弛线性规划 (LP1)。令 x_{ij} 为机器 i 被分配给 j 的时间步数, d_j 为至少有一台机器被分配给 j 的时间步数。

$$(\text{LP1}) \quad \min \quad t$$

$$\text{s.t.} \quad \sum_{i \in M} p_{ij} x_{ij} \geq 1/2 \quad \forall j \in J \quad (1)$$

$$\sum_{j \in J} x_{ij} \leq t \quad \forall i \in M \quad (2)$$

$$\sum_{j \in C_k} d_j \leq t \quad C_k \in C \quad (3)$$

$$0 \leq x_{ij} \leq d_j \quad \forall i, j \quad (4)$$

$$d_j \geq 1 \quad \forall j \quad (5)$$

说明 (LP1) 的几个要点。式 1 强制满足条件 (i)。式 2 限制每台机器的负载，其定义将在下文给出。式 3 限制每条链约束的时间长度。最后，式 4 保证每个作业在至少有一台机器分配给它的 d_j 个时间步内积累质量。令 T^* 为上述 (LP1) 的最优值。

注意，(LP1) 没有约束来禁止来自不同先后约束链的两个作业在同一时间步被安排到同一机器。我们使用术语伪调度表示这种“调度”：不同链的不同作业可以同时被安排到同一机器。

定义 4.1. 长度为 $T \in \mathbb{Z}^+ \cup \infty$ 的伪调度是分配函数集合 $\{f_t : M \rightarrow 2^J \mid 1 \leq t < T+1\}$ 。

因此，伪调度的分配函数可能将一台机器映射到一个作业集合。从这个意义上，伪调度可能不可行；后文说明如何把伪调度转换为合适的无反馈调度时，将解决这一问题。无反馈调度是 f_t 的值为单个元素的伪调度。

定义 4.2. 给定 (有限) 长度为 T 的伪调度 Σ_g ，其分配函数集合为 $\{f_t : M \rightarrow 2^J \mid 1 \leq t < T+1\}$ 。定义机器 i 的负载为 Σ_g 中把某个作业安排到 i 的总次数。形式上，机器 i 的负载为 $\sum_{1 \leq t < T+1} |f_t(i)|$ 。 Σ_g 的负载定义为各机器负载的最大值。

注意，长度为 T 的伪调度可能具有大于 T 的负载。

定理 4.1. 可以在多项式时间内舍入 (LP1) 的最优可行解，得到问题 AccuMass-C 的一个伪调度，其长度和负载都是 $O(\log m)T^*$ 。

证明。显然，(LP1) 是可行的，因为可以为每个作业分配有限步的机器，使其积累质量 $1/2$ 。令 $\{x_{ij}, d_j, t\}$ 为 (LP1) 的一个最优解。（注意， t 等于 T^* 。）主要工作在于舍入过程，即从分数解得到可行整数解，同时避免过度增大 t 。随后说明如何从 (LP1) 的整数解得到伪调度。分两种情形处理。

第一种情形是 $t \geq |J| = n$ 。将每个 x_{ij} 和 d_j 向上舍入，令 $x_{ij}^* = \lceil x_{ij} \rceil$ 和 $d_j^* = \lceil d_j \rceil$ 。由下式得到近似因子为 2 的可行整数解：

$$\sum_{i \in M} p_{ij} x_{ij}^* \geq 1/2 \quad \forall j \in J,$$

$$\sum_{j \in J} x_{ij}^* \leq t + n \leq 2t \quad \forall i \in M,$$

$$\sum_{j \in C_k} d_j^* \leq t + n \leq 2t \quad C_k \in C,$$

$$x_{ij}^* \leq d_j^* \quad \forall i, j.$$

第二种情形是 $t < |J| = n$ 。此时使用网络流理论的结果进行舍入。注意，虽然目标质量为 $1/2$ ，任意小于 $1/2$ 的常数也可以，因为总能把每个变量按比例放大到目标，只损失一个常数

因子。下文会多次使用这样的放大操作。（我们没有尝试优化这些常数。）对给定作业 j ，若 $\sum_{i \in M, x_{ij} \geq 1} p_{ij} x_{ij} \geq 1/4$ ，就可以把这些 x_{ij} 向上舍入到下一个整数。由于 $\lceil x_{ij} \rceil \leq 2x_{ij}$ ，只会使 t 增大 2 倍。因此，只需考虑满足 $\sum_{i \in M, x_{ij} \geq 1} p_{ij} x_{ij} \leq 1/4$ 的作业 j ；这意味着 $\sum_{i \in M, x_{ij} < 1} p_{ij} x_{ij} \geq 1/4$ 。注意， $\sum_{i \in M, p_{ij} < \frac{1}{8m}, x_{ij} < 1} p_{ij} x_{ij} < 1/8$ ，从而有 $\sum_{i \in M, p_{ij} \geq \frac{1}{8m}, x_{ij} < 1} p_{ij} x_{ij} \geq 1/8$ 。

把这些 p_{ij} 分到至多 $B = \lceil \log(8m) \rceil$ 个区间 $(2^{-(k+1)}, 2^{-k}]$ ($k = 0, 1, \dots$) 中。对桶 $b : (2^{-(b+1)}, 2^{-b}]$ ，若 $\sum_{p_{ij} \in \text{桶} b} x_{ij} < 1/32$ ，则移除该桶，不再考虑。注意，全部被移除桶中的 $p_{ij} x_{ij}$ 之和至多为 $1/16$ 。因此，对剩余桶中的 p_{ij} ，仍有 $\sum_{i \in M, p_{ij} \geq \frac{1}{8m}, x_{ij} < 1} p_{ij} x_{ij} \geq 1/16$ 。

对每个作业 j ，都有一个桶 $b_j : (2^{-(b_j+1)}, 2^{-b_j}]$ ，使得 $\sum_{p_{ij} \in \text{桶} b_j} x_{ij} \geq \frac{2^{b_j}}{16B}$ 。

将上述不等式左侧的和记为 D_j 。必要时，将全部 x_{ij} 及其他变量放大 32 倍，使所有 $D_j \geq 1$ 。随后把 D_j 向下舍入为 $\lfloor D_j \rfloor$ 。这些操作只损失常数近似因子。因此，为便于下面的表述，假设 D_j 都是整数，并令 $D = \sum_{j \in J} D_j$ 。

按如下方式构造网络流实例（见图 3）。每个作业 j 对应一个节点，每台机器 i 对应一个节点，另有源节点 u 和汇节点 v 。对每个参与计算 D_j 的 x_{ij} ，添加一条边 (i, j) ；将边 (i, j) 定向为从 j 到 i ，容量为 $\lfloor d_j \rfloor$ 。从每个机器节点 i 添加一条指向 v 的边，容量为 $\lceil 2t \rceil$ 。对每个作业节点 j ，添加从 u 到 j 的边，容量为 D_j 。

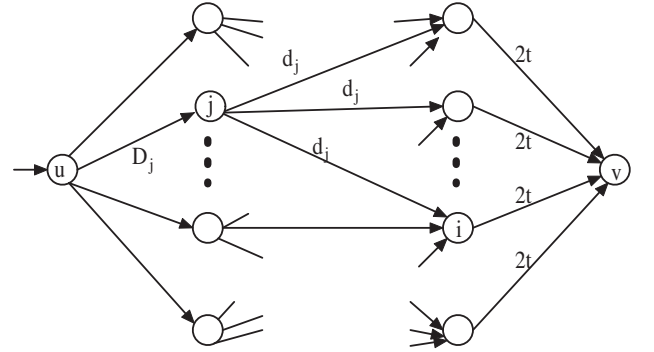


图 3: 对 (LP1) 最优解进行舍入所用的网络流实例

构造之前的论证说明，该网络可以承载源节点 u 需求为 D 的流，其中 x_{ij} 指定了这样一个可行流。 D 实际上就是网络的最大流值（考虑一侧仅有 u 的割）。由 Ford-Fulkerson 定理 [8, 5] 可知，当参数为整数时，例如本实例，存在一个整数可行流。将边 (j, i) 上的整数流值作为舍入解 x_{ij}^* 。此外，所得整数解满足以下关系：

$$\sum_{i \in M} p_{ij} x_{ij}^* \geq \frac{1}{16 \lceil \log(8m) \rceil} \quad \forall j \in J,$$

$$\sum_{j \in J} x_{ij}^* \leq \lceil 2t \rceil \quad \forall i \in M,$$

$$\sum_{j \in C_k} \lfloor d_j \rfloor \leq \lceil 2t \rceil \quad C_k \in C,$$

$$x_{ij}^* \leq \lfloor d_j \rfloor \quad \forall i, j.$$

将全部值放大 $O(\log m)$ 倍，得到整数可行解 $\{\hat{x}_{ij}, \hat{d}_j, \hat{t}\}$ ，其

中 $\hat{t} = O(\log m)T^*$ 。

现在说明如何从该整数解构造伪调度 Σ_s ，使长度和负载都受 $\hat{t} = O(\log m)T^*$ 限制。考虑链 $C_k \in C$ 中的作业 j 。给定 $\hat{x}_{ij}$ ，令 $L_j = \max_i \hat{x}_{ij}$ ， $\psi_j = \sum_{j_0: j_0 \prec j} L_{j_0}$ 。在从时间步 $\psi_j + 1$ 到 $\psi_j + L_j$ 、长度为 L_j 的区间内，把机器分配给 j ，每台机器 i 使用 $\hat{x}_{ij}$ 次。换言之，链 C_k 的分配函数定义如下：对任意作业 j 和机器 i ，若 $\hat{x}_{ij} > 0$ ，则在 $t \in [\psi_j + 1, \psi_j + \hat{x}_{ij}]$ 上有 $f_t^k(i) = \{j\}$ 。可以这样做，是因为每台机器至多被分配给 j 共 L_j 次，而且不同机器可以在同一时间步被分配给 j 。定义每条链 $C_k \in C$ 的 $f_t^k(\cdot)$ 后，将 Σ_s 的分配函数定义为

$$f_t(i) = \cup_{k: C_k \in C} f_t^k(i) \quad \text{其中 } i \in M, t \in [1, \hat{t}].$$

回顾一下，伪调度分配函数的值域是作业集合。定理证明完成。

□

现在建立 AccuMass-C 与 SUU-C 的联系。回顾一下， T^* 是问题 AccuMass-C 写出的 (LP1) 的最优值， T^{OPT} 是问题 SUU-C 的最优调度 Σ 的期望完工跨度。引理 4.2 将用 T^{OPT} 给出 T^* 的界。该引理与定理 4.1 立即得到一个求解 AccuMass-C 的伪调度，其负载和长度都在 T^{OPT} 的 $O(\log n)$ 倍以内。

引理 4.2. $T^* \leq 16T^{\text{OPT}}$. □

证明。下面的线性规划与 (LP1) 相同，只是将 $1/2$ 换成 $1/16$ ，将 t 换成 $2T^{\text{OPT}}$ 。我们说明这个线性规划可行。

$$\begin{aligned} \sum_{i \in M} p_{ij} x_{ij} &\geq 1/16 \quad \forall j \in J \\ \sum_{j \in J} x_{ij} &\leq 2T^{\text{OPT}} \quad \forall i \in M \\ \sum_{j \in C_k} d_j &\leq 2T^{\text{OPT}} \quad C_k \in C \\ x_{ij} &\leq d_j \quad \forall i, j \\ d_j &\geq 1 \quad \forall j \\ x_{ij} &\geq 0 \quad \forall i, j \end{aligned}$$

考虑最优调度 Σ 执行的前 $2T^{\text{OPT}}$ 步。令随机变量 X_{ij} 为 i 被分配给 j 的时间步数，随机变量 Y_j 为至少有一台机器被分配给 j 的总时间步数。由定理 2.2， j 以至少 $1/4$ 的概率在 $2T^{\text{OPT}}$ 步内积累至少 $1/4$ 的质量。这意味着 j 的期望积累质量至少为 $1/16$ 。因此，

$$\sum_{i \in M} p_{ij} \cdot E[X_{ij}] \geq 1/16.$$

由于 Σ 在任意时间步至多把一台机器分配给一个作业，有 $\sum_{j \in J} X_{ij} \leq 2T^{\text{OPT}}$ 。所以，

$$\sum_{j \in J} E[X_{ij}] \leq 2T^{\text{OPT}}.$$

因为只考虑 Σ 的 $2T^{\text{OPT}}$ 步，得到 $\sum_{j \in C_k} Y_j \leq 2T^{\text{OPT}}$ 。显然， $X_{ij} \leq Y_j$ 。取期望可得

$$\sum_{j \in C_k} E[Y_j] \leq 2T^{\text{OPT}}$$

和

$$E[X_{ij}] \leq E[Y_j].$$

因此，对 $i \in M, j \in J$ 取 $x_{ij} = E[X_{ij}]$ ，对 $j \in J$ 取 $d_j = E[Y_j]$ ，就构成该线性规划的一个解。将此解放大 8 倍，得到 (LP1) 的一个解。这意味着 (LP1) 能够达到 t 值 $16T^{\text{OPT}}$ 。因此证明了 $T^* \leq 16T^{\text{OPT}}$ 。引理证明完成。

□

定理 4.3. 可以在多项式时间内计算长度和负载都受 $O(\log m) \cdot T^{\text{OPT}}$ 限制的伪调度，使其满足：(i) 每个作业 j 积累至少 $1/2$ 的质量；(ii) 若 $j_1 \prec j_2$ ，则只有在 j_1 积累 $1/2$ 的质量后， j_2 才能开始积累质量。□

本节其余部分说明如何把定理 4.3 得到的伪调度转换为可行调度。由定理 4.3，可以计算长度为 $O(\log m) \cdot T^{\text{OPT}}$ 的伪调度 Σ_s ，使每个作业积累至少 $1/2$ 的质量，从而具有至少 $\frac{1}{2e}$ 的成功概率。此外，若 $j_1 \prec j_2$ ，则在 j_1 积累质量 $1/2$ 前，不会给 j_2 分配机器。接下来分两步把 Σ_s 转换为可行的无反馈调度 Σ_o 。

1. 使用 [19, 27] 的随机延迟技术，适当地推迟每条链执行的起始时间步，得到新的伪调度 $\Sigma_{s,1}$ ，其中任意机器在任意时间步安排的作业数为 $O(\frac{\log(n+m)}{\log \log(n+m)})$ 。还可以用 [22, 25, 27] 的技术对该随机调度去随机化。随后将 $\Sigma_{s,1}$ “展平”，得到无反馈调度 $\Sigma_{o,1}$ ，使调度长度增加 $O(\frac{\log(n+m)}{\log \log(n+m)})$ 倍。
2. 为得到最终的无反馈调度 Σ_o ，取上述无反馈调度 $\Sigma_{o,1}$ ，将每个时间步的机器分配复制 $O(\log n)$ 次，使全部作业以高概率完成。

下面详细描述把伪调度转换为可行无反馈调度的两个步骤。因为第二步较简单，先介绍它。

调度复制：首先将 $\Sigma_{o,1}$ 的每个时间步复制 $\sigma = 16 \log n$ 倍，得到另一个无反馈调度 $\Sigma_{o,2}$ 。更精确地，令 T 为 $\Sigma_{o,1}$ 的长度， $g_t(\cdot)$ 为 $\Sigma_{o,1}$ 的分配函数。 $\Sigma_{o,2}$ 的分配函数 $f_t(\cdot)$ 定义如下：对任意 $t \in [1, \sigma \cdot T]$ ，有 $f_t(\cdot) = g_\tau(\cdot)$ ，其中 $\tau = \lfloor \frac{t-1}{\sigma} \rfloor + 1$ 。注意，若 $\Sigma_{o,1}$ 可以用相对于输入大小为多项式的空间表示，如“延迟”步骤中将证明的那样，则 $\Sigma_{o,2}$ 也可以。

再定义一个长度为 n 的无反馈调度 $\Sigma_{o,3}$ 。按照先后约束对作业进行拓扑排序，例如将各条先后约束链依次连接，令 $j_1, \dots, j_n$ 为排序后的作业。 $\Sigma_{o,3}$ 的分配函数 $h_t(\cdot)$ 定义为： $\forall i \in M, h_t(i) = j_t$ ，其中 $1 \leq t \leq n$ 。所需的最终无反馈调度为 $\Sigma_o = \Sigma_{o,2} \circ \Sigma_{o,3}$ 。换言之，无反馈调度 Σ_o 先执行复制后的 $\Sigma_{o,1}$ ，随后在每个时间步把全部机器分配给某个作业。

现在分析 Σ_o 的期望完工跨度。若全部作业在时间步 σT 以内成功完成，期望完工跨度至多为 σT 。这一事件未发生的概率至多为 $n(1 - \frac{1}{e})^\sigma < 1/n^2$ 。还要注意，从时间步 $\sigma T + 1$ 开始， Σ_o 在每个时间步都把全部机器周期性地分配给单个作业（这是 $\Sigma_{o,3}$ 的作用，周期长度为 n ）。若把全部机器都分配给某个作业，则其完成所需时间步数的期望至多为 T^{OPT} 。由于这里周期性地给任意固定作业分配机器，完成任意固定作业平均至多需要 (nT^{OPT}) 步。因此，使用时间步 σT 之后的分配函数，完成全部作业平均至多需要 $n^2 T^{\text{OPT}}$ 步。于是， Σ_o 的期望完工跨度至多为

$$(1 - 1/n^2) \sigma \cdot T + 1/n^2 \cdot (\sigma \cdot T + n^2 T^{\text{OPT}}).$$

稍后将证明， $T = O(\log m \frac{\log(n+m)}{\log \log(n+m)}) \cdot T^{\text{OPT}}$ ，且 $\sigma = 16 \log n$ 。

因此， Σ_o 的期望完工跨度是 $O(\log n \log m \frac{\log(n+m)}{\log \log(n+m)}) \cdot T^{\text{OPT}}$ 。

将伪调度 Σ_s 转换为无反馈调度：现在处理定理 4.3 计算的伪调度 Σ_s 。尚不可行的情形，即某台机器在同一时间步被分配给不止一个作业。我们说明，可以把 Σ_s 转换为无反馈调度 $\Sigma_{o,1}$ ，仅损失 $O(\frac{\log(n+m)}{\log \log(n+m)})$ 因子。

令 Π_{max} 为 Σ_s 的负载, 即分配给任意机器的作业数量的最大值。Shmoys, Stein 和 Wein 关于作业车间调度的结果 [27, 引理 2.1] 表明, 若把每条链的起始时间步延迟一个从 $[0, \Pi_{max}]$ 中独立、均匀选出的整数, 则所得伪调度在任意时间步、任意机器上安排的作业数至多为 $O(\frac{\log(n+m)}{\log \log(n+m)})$ 。下面解释延迟的含义。回顾定理 4.1 证明的最后一段: 先为每条约束链 $C_k \in C$ 指定函数 f_t^k , 再把 Σ_s 的分配函数定义为 $f_t = \cup_k f_t^k$ 。假设链 C_k 延迟 ϕ_k , 则链 C_k 的分配函数 g_t^k 修改如下: $\forall i \in M$, 若 $t \leq \phi_k$, $g_t^k(i) = \emptyset$; 否则 $g_t^k(i) = f_{t-\phi_k}^k(i)$ 。整个调度的分配函数定义为 $f_t = \cup_k g_t^k$ 。为使叙述自成一统, 下面概述 $O(\frac{\log(n+m)}{\log \log(n+m)})$ 界的论证。

固定时间步 t 和机器 i 。令 $p = \Pr$ 在时间步 t 中, 至少 τ 个处理单位被安排到机器 i 上。注意, 一个作业 j 可以在多个时间步得到安排, 而且每个作业均为单位时间步; 这等于作业 j 具有多个处理单位。选择这 τ 个处理单位的方式至多有 $\binom{\Pi_{max}}{\tau}$ 种。关注某一种 τ 个单位的选择。若这些单位来自不同链, 则它们都被安排到时间步 t 的概率至多为 $(\frac{1}{\Pi_{max}})^\tau$, 因为延迟在 $[0, \Pi_{max}]$ 中独立、均匀选择。否则, 概率为 0, 因为我们的伪调度绝不会在同一时间步把同一链的两个单位分配给同一机器。因此,

$$\begin{aligned} p &\leq \binom{\Pi_{max}}{\tau} \left(\frac{1}{\Pi_{max}} \right)^\tau \\ &\leq \left(\frac{e\Pi_{max}}{\tau} \right)^\tau \left(\frac{1}{\Pi_{max}} \right)^\tau \\ &\leq \left(\frac{e}{\tau} \right)^\tau \end{aligned}$$

若 $\tau = \alpha \frac{\log(n+m)}{\log \log(n+m)}$, 则 $p < (n+m)^{-(\alpha-1)}$ 。令 L_{max} 为 Σ_s 中最长链的长度。任意机器在任意时间步被分配至少 $\alpha \frac{\log(n+m)}{\log \log(n+m)}$ 个作业的概率, 受 $m(\Pi_{max} + L_{max})(n+m)^{-(\alpha-1)}$ 限制。暂时假设 T^{OPT} 受 $(n+m)$ 的多项式限制, 该假设稍后将去除; 此时 $\Pi_{max} + L_{max}$ 也受 $(n+m)$ 的多项式限制。如果选择足够大的 α , 则以高概率, 任意机器在任意时间步安排的作业数都不超过 $\alpha \frac{\log(n+m)}{\log \log(n+m)}$ 。

Shmoys, Stein 和 Wein [27] 还利用 [23, 24, 22] 的结果对算法去随机化, 使任意机器同时安排的作业数为 $O(\log(n+m))$ 。Schmidt, Siegel 和 Srinivasan [25] 给出另一种去随机化策略, 得到与随机算法匹配的冲突界, 即任意机器上同时安排 $O(\frac{\log(n+m)}{\log \log(n+m)})$ 台机器。将这一去随机化的伪调度记为 $\Sigma_{s,1}$, 其长度至多为 Σ_s 的两倍。根据定理 4.3, Σ_s 的长度是 $O(\log m) \cdot T^{OPT}$, 因此可以“展平” $\Sigma_{s,1}$, 得到长度为 $O(\log m \frac{\log(n+m)}{\log \log(n+m)}) \cdot T^{OPT}$ 的无反馈调度 $\Sigma_{o,1}$, 其中每台机器在任意时间步都只分配给一个作业。随机延迟技术最早出现在 [19] 对作业车间调度问题的研究中。

缩小 T^{OPT} : 现在处理 T^{OPT} 不一定受 $(n+m)$ 的多项式限制的问题。使用 [27, 第 3.1 节] 的一个技巧。考虑定理 4.3 计算的伪调度 Σ_s 。对于每个作业 j , 令 l_{ij} 为机器 i 被分配给 j 的时间步数, L_j 为 $\max_i l_{ij}$, 将 $\max_j L_j$ 记为 L 。所有机器都在长度为 L_j 的窗口内分配给 j 。令 $\beta = nm$ 。将每个 l_{ij} 向下舍入到最近的 $\frac{L}{\beta}$ 的倍数, 记该值为 l'_{ij} 。因此, 可以把 l'_{ij} 视为 $\{0, \dots, \beta\}$ 中的整数。新问题的调度可以直接缩放回具有真实值 l'_{ij} 的调度。由于 $\beta = nm$, 此时调度的长度和负载实际上都受 $(n+m)$ 的多项式限制。因此, 前面对随机延迟和去随机化的讨论现在成立。令 Σ' 为所得可行无反馈调度, 其长度受 $O(\log m \frac{\log(n+m)}{\log \log(n+m)}) T^{OPT}$ 限制, 负载受 $O(\log m) T^{OPT}$ 限制。为得到让每个作业积累 1/2 质量的可行无反馈调度 $\Sigma_{o,1}$, 向 Σ' 中插入 $(l_{ij} - l'_{ij})$ 个处理单位。可以按保持先后约束的

方式插入: 若 $j_1 \prec j_2$, 则在 j_1 积累 1/2 的质量前, 不能给 j_2 分配机器。每次插入使 Σ' 的长度增加 $\leq \frac{L}{nm}$, 而插入次数至多为 nm , 因此调度长度至多增加 L 。机器负载与舍入前相同。注意, L 受 Π_{max} 限制, 后者为 $O(\log m) T^{OPT}$ 。于是, 得到长度为 $O(\log m \frac{\log(n+m)}{\log \log(n+m)}) T^{OPT}$ 的可行无反馈调度 $\Sigma_{o,1}$, 其中每个作业都积累常数质量。最后, 使用本节前面讨论的复制技术得到所需调度。

定理 4.4. 对于问题SUU-C, 存在多项式时间算法, 计算一个无反馈调度, 其期望完工跨度在最优值的 $O(\log m \log n \frac{\log(n+m)}{\log \log(n+m)})$ 倍以内。□

对于独立作业, 即问题SUU-C 的约束 C 为空时, 可以为无反馈调度证明一个比 §3 末尾结果略好的界。

定理 4.5. 对于问题SUU-I, 存在多项式时间算法, 计算一个无反馈调度, 其期望完工跨度在最优值的 $O(\log n \cdot \log(\min\{n, m\}))$ 倍以内。□

证明. 令 (LP2) 为从 (LP1) 去除约束 3、4、5 得到的线性规划, T_2^* 为 (LP2) 的最优值。首先说明, 可以舍入 (LP2) 的最优可行解, 得到问题 AccuMass-C 的一个无反馈调度, 其长度及负载均为 $O(\log(\min\{n, m\})) \cdot T_2^*$ 。

对于问题SUU-I, AccuMass-C 的条件 (ii) 没有实际约束, 因此在线性规划中无需约束 3、4、5。定理 4.1 证明中的舍入引入 $O(\log m)$ 倍放大。若 $m \geq n$, 可以对舍入过程作更好的分析。因为 (LP2) 有 $n+m$ 个非平凡约束, 任意基本可行解至多具有 $n+m$ 个非零值 [2, 26]。对于一个最优且基本可行的解 $\{x_{ij}, t\}$, 不失一般性, 可以假设任意机器 i 都存在某个 j 满足 $x_{ij} > 0$; 否则, 可以在 (LP2) 中移除该机器。因此, 至少具有两个 $x_{ij} > 0$ 的机器 i 的数量至多为 n 。舍入 x_{ij} 时, 只需考虑至少具有两个 $x_{ij} > 0$ 的机器 i 。此时, 定理 4.1 证明中的相同舍入过程引入 $O(\log n)$ 倍放大, 因为对每个作业只需考虑 $O(\log n)$ 个桶。

因此, 可以得到整数可行解 $\{\hat{x}_{ij}, \hat{t}\}$, 其中 $\hat{t} = O(\log(\min\{n, m\})) \cdot T_2^*$ 。此外, 由 $\{\hat{x}_{ij}, \hat{t}\}$ 可以构造问题 AccuMass-C 的可行无反馈调度, 其长度和负载为 $\hat{t} = O(\log(\min\{n, m\})) \cdot T_2^*$ 。这是因为, 按式 2, 每台机器的负载受 $\hat{t}$ 限制, 而且作业独立。因此, 可以安排机器分配, 使任意机器在任意时间步至多处理一个作业。

由此得到一个无反馈调度, 使每个作业在至多为最优值 $O(\log(\min\{n, m\}))$ 倍的时间内积累常数质量。再应用调度复制步骤, 就得到所需的界。

□

4.2 树形先后约束

针对树形先后约束的算法使用 [17] 的技术。他们将 [27] 对链式先后约束下不相关并行机器调度的工作扩展到树形先后约束: 把有向森林分解为 $O(\log n)$ 组链。为陈述该结果, 先引入 [17] 的记号。给定 DAG $G(V, E)$, 令 $d_{in}(u)$ 和 $d_{out}(u)$ 分别为 G 中 u 的入度和出度。 G 的一个链分解, 是将其顶点集划分为子集 $B_1, \dots, B_\lambda$ (称为块), 满足: (i) 每个块 B_i 诱导的子图是一组顶点互不相交的有向链; (ii) 对任意 $u, v \in V$, 若 $u \in B_i$ 是 $v \in B_j$ 的祖先, 则或者 $i < j$, 或者 $i = j$ 且 u 与 v 属于 B_i 的同一条有向链; (iii) 若 $d_{out}(u) > 1$, 则 u 的所有出邻居都不与 u 同块。DAG 的链宽定义为能够将该 DAG 链分解为 λ 个块的最小 λ 值。下面陈述分解结果。

引理 4.6 ([17], 引理 1). 任意底层无向图为森林的 DAG 都有宽度为 γ 的链分解, 其中 $\gamma \leq 2(\lceil \log n \rceil + 1)$ 。该分解可以在多项式时间内计算。

利用引理 4.6, 将给定有向森林分解为至多 $\gamma = O(\log n)$ 个块, 并在每个块内应用链情形的算法 (定理 4.4)。任意子图 (作业子集) 的最优期望完工跨度都是整个图 (全部作业) 的下界, 因此该方法额外损失一个 $\log n$ 因子。由此得到:

定理 4.7. 对于问题SUU, 若依赖图 C 为有向森林, 则存在多项式时间算法, 计算一个无反馈调度, 其期望完工跨度在最优值的 $O(\log m \log^2 n \frac{\log(n+m)}{\log \log(n+m)})$ 倍以内。

当先后约束构成一组出树 (边从根向外定向的有根树) 或入树 (类似定义) 时, 再遵循 [17] 的思想, 可以得到更好的近似算法。具体而言, 将出树或入树分解为 $O(\log n)$ 个块; 随后, 把每条链随机延迟一个从 $[0, O(\Pi_{\max}/\log n)]$ 中均匀选择的时间步数 (可以在多项式时间内去随机化); 再证明, 以高概率, 任意机器同时安排的作业数至多为 $O(\log n)$ 。

定理 4.8. 对于问题SUU, 若依赖图 C 为出树或入树的集合, 则存在多项式时间算法, 计算一个无反馈调度, 其期望完工跨度在最优值的 $O(\log m \log^2 n)$ 倍以内。

5. 开放问题

本文针对几类特殊依赖图, 为不确定性下的多处理器调度问题给出多对数近似算法。我们认为这些界并不紧; 具体而言, 猜想更细致的分析可以使每种情形的近似比改进一个 $O(\log n)$ 因子。此外, 为更一般的依赖类别得到近似算法, 以及研究该调度问题的在线版本, 也是值得探索的方向。

6. 参考文献

- [1] D. Applegate and B. Cook. A computational study of the job-shop scheduling problem. *ORSA Journal of Computing*, 3(2):149–156, 1991.
- [2] D. Bertsimas and J. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997.
- [3] H. Chernoff. A measure of the asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Annals of Mathematical Statistics*, 23:493–509, 1952.
- [4] F. Chudak and D. Shmoys. Approximation algorithms for precedence-constrained scheduling problems on parallel machines that run at different speeds. *Journal of Algorithms*, 30, 1999.
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press and McGraw-Hill Book Company, Cambridge, MA, second edition, 2001.
- [6] A. Fernandez, R. Armacost, and J. Pet-Edwards. A model for the resource constrained project scheduling problem with stochastic task durations. In *7th Industrial Engineering Research Conference Proceedings*, 1998.
- [7] A. Fernandez, R. Armacost, and J. Pet-Edwards. Understanding simulation solutions to resource constrained project scheduling problems with stochastic task durations. *Engineering Management Journal*, 10(4):5–13, 1998.
- [8] L. R. Ford, Jr. and D. R. Fulkerson. *Flows in Networks*. Princeton University Press, Princeton, 1962.
- [9] I. Foster and C. Kesselman, editors. *The Grid: Blueprint for a New Computing Infrastructure*. Morgan Kaufmann, San Francisco, CA, 2nd edition, 2004.
- [10] M. R. Garey and D. S. Johnson. *Computers and Intractability: A guide to the theory of NP-completeness*. W. H. Freeman, San Francisco, 1979.
- [11] A. Goel and P. Indyk. Stochastic load balancing and related problems. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS)*, 1999.
- [12] R. L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal (BSTJ)*, 45:1563–1581, 1966.
- [13] L. Hall. Approximation algorithms for scheduling. In D. Hochbaum, editor, *Approximation Algorithms for NP-hard Problems*, PWS Publishing Company, 1997.
- [14] W. Herroelen and R. Leus. Project scheduling under uncertainty: Survey and research potentials. *European Journal of Operational Research*, 165(2):289–306, 2005.
- [15] W. Hoeffding. On the distribution of the number of successes in independent trials. *Annals of Mathematical Statistics*, 27:713–721, 1956.
- [16] J. Kleinberg, Y. Rabani, and E. Tardos. Allocating bandwidth for bursty connections. *SIAM Journal on Computing*, 30, 2000.
- [17] V. Kumar, M. Marathe, S. Parthasarathy, and A. Srinivasan. Scheduling on unrelated machines under tree-like precedence constraints. In *International Workshop on Approximation Algorithms for Combinatorial Optimization*, 2005.
- [18] E. L. Lawler, J. K. Lenstra, A. R. Kan, and D. B. Shmoys. Sequencing and scheduling: Algorithms and complexity. Technical Report BS-R8909, Centre for Mathematics and Computer Science., Amsterdam, 1991.
- [19] F. T. Leighton, B. M. Maggs, and S. Rao. Packet routing and job-shop scheduling in $O(\text{congestion} + \text{dilation})$ steps. *Combinatorica*, 14(2):167–186, 1994.
- [20] J. Lenstra, D. Shmoys, and E. Tardos. Approximation algorithms for scheduling unrelated parallel machines. *Mathematical Programming*, 46, 1990.
- [21] G. Malewicz. Parallel scheduling of complex dags under uncertainty. In *Proceedings of the 17th annual ACM symposium on Parallelism in algorithms and architectures*, pages 66–75, Las Vegas, Nevada, USA, 2005.
- [22] P. Raghavan. Probabilistic construction of deterministic algorithms: Approximating packing integer programs. *Journal of Computer and System Sciences*, 37, 1988.
- [23] P. Raghavan and C. Thompson. Provably good routing in graphs: Regular arrays. In *ACM Symposium on Theory of Computing (STOC)*, 1985.
- [24] P. Raghavan and C. Thompson. Randomized rounding: A technique for provably good algorithms and algorithmic proofs. *Combinatorica*, 7, 1987.
- [25] J. Schmidt, A. Siegel, and A. Srinivasan. Chernoff-hoeffding bounds for applications with limited independence. *SIAM Journal on Discrete Mathematics*, 8, 1995.
- [26] A. Schrijver. *Theory of linear and integer programming*. Interscience Series in Discrete

Mathematics and Optimization. Wiley, 1986.

- [27] D. Shmoys, C. Stein, and J. Wein. Improved approximation algorithms for shop scheduling problems. *SIAM Journal on Computing*, 23, 1994.
- [28] M. Skutella. Convex quadratic and semidefinite programming relaxations in scheduling. *Journal of the Association for Computing Machinery (JACM)*, 48(2):206–242, 2001.
- [29] M. Skutella and M. Uetz. Scheduling precedence-constrained jobs with stochastic processing times on parallel machines. In *Proceedings of the twelfth annual ACM-SIAM symposium on Discrete algorithms (SODA)*, pages 589–590, Washington, D.C., US, 2001.